

F24-046-D-Nether'sGate

Project Team

Muhammad Taha Qaiser	21K-4779
Qalander Aziz	21I-1577
Khubab Ahmed	21I-1567

Session 2021-2025

Supervised by

Mr. Muhammad Abdullah Abid



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

October, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1: Application Construction	3
1.3.2	Module 2: Obfuscation Techniques	3
1.3.3	Module 3: Evasion Mechanisms	3
1.3.4	Module 4: User Interface and Compiler Control	4
1.4	User Classes and Characteristics	4
2	Project Requirements	7
2.1	Use-case/Event Response Table/Storyboarding	7
2.2	Functional Requirements	8
2.2.1	Module 1: Application Construction	8
2.2.2	Module 2: Obfuscation Techniques	8
2.2.3	Module 3: Evasion Mechanisms	8
2.2.4	Module 4: User Interface and Compiler Control	9
2.3	Non-Functional Requirements	9
2.3.1	Reliability	9
2.3.2	Usability	10
2.3.3	Performance	10
2.3.4	Security	10
3	System Overview	11
3.1	Architectural Design	11
3.2	Design Models	13
	References	17

List of Figures

2.1	Nether's Gate Use Case Diagram	7
3.1	Nether's Gate Box and Line Architecture Diagram	12
3.2	Nether's Gate Layered Architecture Diagram	13
3.3	Nether's Gate Activity Diagram	14
3.4	Nether's Gate DFD Level 0 Diagram	15
3.5	Nether's Gate DFD Level 1 Diagram	15
3.6	Nether's Gate System Level Sequence Diagram	16

List of Tables

1.1 User Classes and Characteristics 5

Chapter 1

Introduction

This document outlines the requirements for the Nether's Gate Malware Obfuscator, a tool designed for automating the obfuscation and evasion techniques used by cybersecurity professionals [4, 5]. The intended audience for this document includes several types of readers:

1. **Developers:** Responsible for implementing the functionalities of the obfuscator, including code compilation, obfuscation methods, and the graphical user interface.
2. **Project Managers:** Oversee the development process, ensuring that the product meets specified requirements, remains on schedule, and stays within budget.
3. **Testers:** Validate that the system meets functional and non-functional requirements, conducting tests to ensure the obfuscator works as intended.
4. **Documentation Writers:** Create user guides, technical documentation, and other materials to help end-users understand and operate the system.
5. **Users:** Cybersecurity professionals and academic researchers who will use the system to obfuscate and test malware samples in controlled environments.

1.1 Problem Statement

The rapid advancement of detection systems like Antivirus (AV), Endpoint Detection and Response (EDR), and Extended Detection and Response (XDR) has made it increasingly difficult for cybersecurity professionals to bypass these defenses during red-team operations and penetration testing [3, 6]. Current tools for malware obfuscation often fall short, being recognized by modern security systems, thus diminishing their effectiveness. Manual application of obfuscation techniques is time-consuming and limits the number

of operations that can be conducted. Furthermore, the existing solutions lack flexibility and scalability to adapt to evolving detection techniques.

To address this, Nether's Gate Malware Obfuscator aims to automate advanced obfuscation and evasion techniques, transforming C++ malware source code into an undetectable malware executable file. The system will also allow hackers, security specialists, and other interested parties to penetrate and probe computer security barriers, study how they work, and perform aggressive security countermeasures more effectively. The tool will be designed to be highly adaptive and flexible for various levels of users from developers to novice cybersecurity researchers because the software will come equipped with a graphical user interface where users will even have an option to select their own obfuscation and evasion techniques.

1.2 Scope

The Nether's Gate Malware Obfuscator is a tool which is created to facilitate the bottling up of C++ malware code to an unrecognizable form of an executable file through the help of evasion. The primary operation encompasses the choice of the distinction methods including variable encryption, full code encryption, garbage code and import obfuscation. However, it also supports such techniques as API unhooking, control flow space manipulation, metamorphism, and sandboxing so that malware escapes any AV, EDR, and XDR.

The project is specifically confined to offer a feasible, responsive platform to security practitioners, academician, and red team performers. It will come with a GUI where users can select the contextual obfuscation methods, evasion strategies and compile the input source code to executable. The system is built to support obfuscation and evasion of malware and it is designed to enhance its capability to escape from the security mechanisms, and it also instructs the assailant about the efficiency of its camouflage and escape tactics in real time basis.

1.3 Modules

This segment of the paper explains the various components of the Nether's Gate Malware Obfuscator. They are as follows Each of the modules is provided to carry out specific functions helpful for the system as a whole.

1.3.1 Module 1: Application Construction

This is a module that is charged with the function of compiling the source code in C++ to a malware executable file. It affirms that the application working can be modified structurally without hindering its performance.

1. Compile source code into an executable format.
2. Apply structural modifications without altering the core behavior of the source code.
3. Parse the important attributes of code from source code.

1.3.2 Module 2: Obfuscation Techniques

This module handles a variety of obfuscation techniques that are applied to the input source code to make it more difficult to reverse-engineer and analyze.

1. Variable encryption to hide variable names and values.
2. Full code encryption to prevent static analysis.
3. Garbage code insertion to confuse reverse engineers.
4. Garbage data insertion to confuse reverse engineers.
5. Import obfuscation to obscure external library and API calls.

1.3.3 Module 3: Evasion Mechanisms

This module implements evasion techniques designed to prevent malware detection by dynamic analysis tools, AV, and EDR systems.

1. API unhooking to modify system calls and bypass security monitoring.
2. Metamorphism to continuously alter code structure without changing functionality.
3. Flow obfuscation to confuse behavioral analysis systems.
4. Sandbox evasion to detect and bypass sandbox environments.

1.3.4 Module 4: User Interface and Compiler Control

This module provides users with a graphical user interface (GUI) for selecting obfuscation, evasion techniques and controlling the compilation process.

1. User interface for selecting obfuscation and evasion techniques.
2. Compile the code and produce an executable file.
3. Real-time feedback on the effectiveness of the obfuscation and evasion techniques.

1.4 User Classes and Characteristics

This section identifies the different user classes that will interact with the Nether's Gate Malware Obfuscator and outlines their pertinent characteristics.

User Class	Description
Cybersecurity Professionals	Cybersecurity professionals, such as red-team operators and penetration testers, will use the obfuscation and evasion techniques to bypass security defenses and assess the effectiveness of security solutions like AV, EDR, and XDR. They are highly skilled individuals, familiar with malware analysis and evasion techniques, and they require a tool that can automate the creation of undetectable malware for testing purposes. They are expected to use the obfuscation and evasion techniques for malware to use in controlled environments to simulate real-world attacks.
Academic Researchers	The tool will specifically be used by academic researchers whose area of specialization involves analysis and detection of malware. They shall learn how current security mechanisms deal with disguised malware infection and how detection can be enhanced. From these users, they are expected to have some level of knowledge of malware development and detection at an intermediate to advance level.
Security Companies	Those security companies providing Offensive Security services will employ the tool during Penetration Testing process where they inject, then use malware to conduct ‘tests’ securely. The unknown attackers will use this tool to generate unique malware versions based upon the obfuscation and evasion procedures to check the effectiveness of the client’s protective systems. This tool is important to these companies in order to make it feasible to implement at a large company level.
Government Agencies	The planned tool will be beneficial for the government agencies involved in the cybersecurity operations concerning sponsorship by assessing the qualities of APTs besides other samples of malware attacks. These agencies will target to utilize it in training known as red-team as well as in national defense operations. They must comply with legal and ethical requirement and the tool should be limited for use in secured environment.
Developers	Developers of the tool will be responsible for maintaining and improving the functionality of the tool. They will focus on implementing new obfuscation techniques, evasion techniques, fixing bugs, and ensuring the tool adapts to evolving security defenses. These users will have a deep understanding of C++ and software obfuscation, evasion techniques.
Testers	Testers will evaluate the system’s performance, usability, and effectiveness in obfuscating and evasion form of executable file. They will conduct tests to verify that the tool meets its functional and non-functional requirements, ensuring it performs as expected across various scenarios. Testers will focus on both security testing and quality assurance.

Table 1.1: User Classes and Characteristics

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the Nether's Gate Malware Obfuscator project.

2.1 Use-case/Event Response Table/Storyboarding

Depending on the nature of this project, a use case diagram may be the most suitable technique for representing the interactions between users and the system. This system focuses on providing security professionals the ability to test obfuscation and evasion techniques [2]. Below is a brief outline of the use cases for the system:

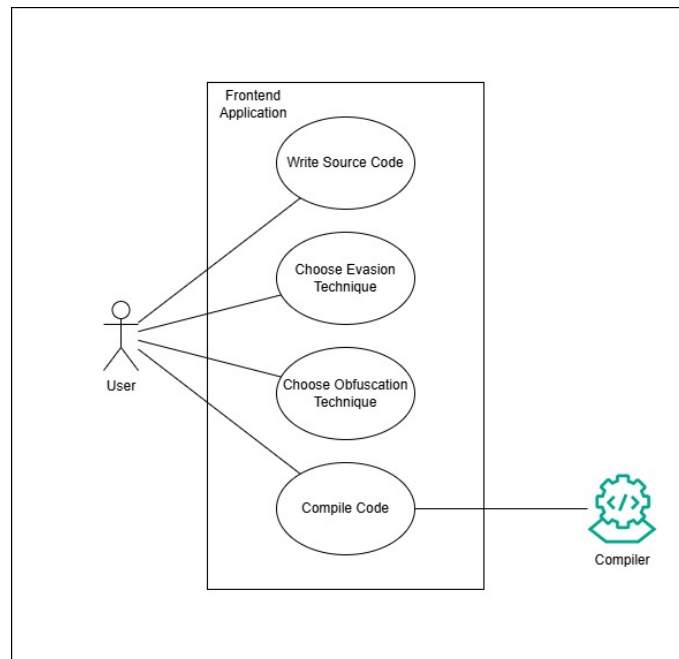


Figure 2.1: Nether's Gate Use Case Diagram

2.2 Functional Requirements

This section describes the functional requirements of the system expressed in the natural language style. This section is typically organized by feature as a system feature name and specific functional requirements associated with this feature. It is just one possible way to arrange them. Other organizational options include arranging functional requirements by use case, process flow, mode of operation, user class, stimulus, and response depend on what kind of technique has been used to understand functional requirements. Hierarchical combinations of these elements are also possible, such as use cases within user classes.

This section outlines the functional requirements for each module in the system.

2.2.1 Module 1: Application Construction

Following are the requirements for module 1:

1. **FR1:** The system shall allow users to provide C++ source files
2. **FR2:** The system shall output obfuscated source code files before compilation
3. **FR3:** The system shall allow users to select different types of obfuscation techniques on their own choice.
4. **FR4:** The system shall allow users to select different types of evasion techniques on their own choice.

2.2.2 Module 2: Obfuscation Techniques

Following are the requirements for module 2:

1. **FR1:** The system shall allow users to apply variable encryption to the source code.
2. **FR2:** The system shall provide the option to insert garbage code into the source code.
3. **FR3:** The system shall allow users to apply advanced obfuscation techniques, such as those described in the ADVobfuscator library [1].

2.2.3 Module 3: Evasion Mechanisms

Following are the requirements for Module 3:

1. **FR1:** The system shall allow users to implement API unhooking techniques.
2. **FR2:** The system shall provide control flow obfuscation techniques to alter execution paths.
3. **FR3:** The system shall detect sandbox environments and apply sandbox evasion techniques.

2.2.4 Module 4: User Interface and Compiler Control

Following are the requirements for Module 4:

1. **FR1:** The system shall provide a graphical user interface for selecting obfuscation techniques.
2. **FR2:** The system shall compile obfuscated malware source code.
3. **FR3:** The system shall generate real-time feedback on the effectiveness of selected obfuscation techniques.

2.3 Non-Functional Requirements

This section outlines the non-functional requirements of the system, focusing on system qualities like reliability, usability, performance, and security.

2.3.1 Reliability

Requirements about how often the software fails. The measurement is often expressed in MTBF (mean time between failures). The definition of a failure must be clear. Be sure to specify the consequences of software failure, how to protect from failure, a strategy for error detection, and a strategy for correction.

The system must have high availability and should ensure that it does not fail during critical operations.

- The system should provide a Mean Time Between Failures (MTBF) of at least 1,000 hours.
- In the case of failure, the system should recover without loss of data.

2.3.2 Usability

It is important that obfuscator is usable meaning that a large group of users should be able to interact with the obfuscator effectively.

- The user interface needs to be intuitive, and should be simple enough that cybersecurity professionals and researchers can understand it with little or no training.
- **USE-1:** Users should be able to use obfuscation techniques with no more than three clicks from anywhere on the main interface.

2.3.3 Performance

Large files have to be handled by the system with great efficiency; tasks must be completed in acceptable time frames.

- **PER-1:** For files up to 10,000 lines of code, we require the system to compile malware in 5 minutes.

2.3.4 Security

To ensure the system is not first used maliciously outside of a testing environment, the system must have strong security measures in place.

- We do not want the user inputs or code uploads to be accessible to anyone but you.
- Even for users, it should require an authentication.

Chapter 3

System Overview

In this chapter, we give a complete picture of the functionality, context, and design behind Nether's Gate, a malware obfuscator project. At its core, the system takes user provided source code, applies evasion and obfuscation techniques, compiles the transformed code to an executable. The design follows a procedural approach by way of decomposing the system into a collection of interacting modules that together perform a specific functionality in the code transformation and compilation process.

3.1 Architectural Design

The system's modular architecture ensures that various functionalities are separated and handled efficiently, following principles outlined by Pressman [4] and Sommerville [5]. The next approach is into the key modules of the system that interact together to accomplish the required work.

In the early stages, the system is represented in terms of a Box and Line Diagram to represent simple architectural design of components of the system and their interactions, which results in a simplified view of the system. It provides a way of understanding the system's structure, in principle, without going into details.

In light of that, we move to a more formal architectural representation later through a Layered Architecture model. This system has separate architectures at the layers, and separate layers to logically separate the user interaction, core logic, and storage of data, with a clear flow of the operations between user input and final output.

Box and Line Architecture Diagram The Box and Line diagram is employed to represent the main modules of the system and their first level dependencies. Initially, this diagram shows a high level view of the system, each box representing a core component and the lines connecting or flowing between them.

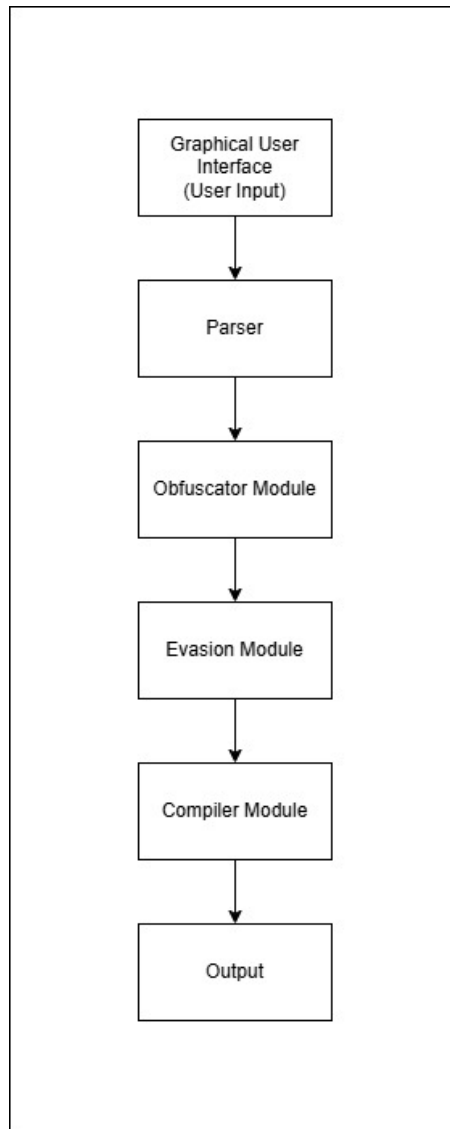


Figure 3.1: Nether's Gate Box and Line Architecture Diagram

This diagram shows the basic flow from the User Input to the Output through the obfuscation and evasion process followed by compilation.

Layered Architecture Diagram

A Layered Architecture pattern is applied to further decompose the system into layers. With this layered approach we have a logical separation of concerns; each layer performs a separate function, and the other layers interact to get the complete system functionality.

1. **Presentation Layer:** This is the layer in which use interacts with the system. The user supplies the source code, chooses the obfuscation and evasion techniques and sees the compilation results.
2. **Application Layer:** This layer is where the business logic of the system is done.

It is compiled after taken the obfuscated, and applying evasion techniques on the code.

3. **Data Layer:** This layer stores the temporary code file, both of source input and of the compiled executable files.

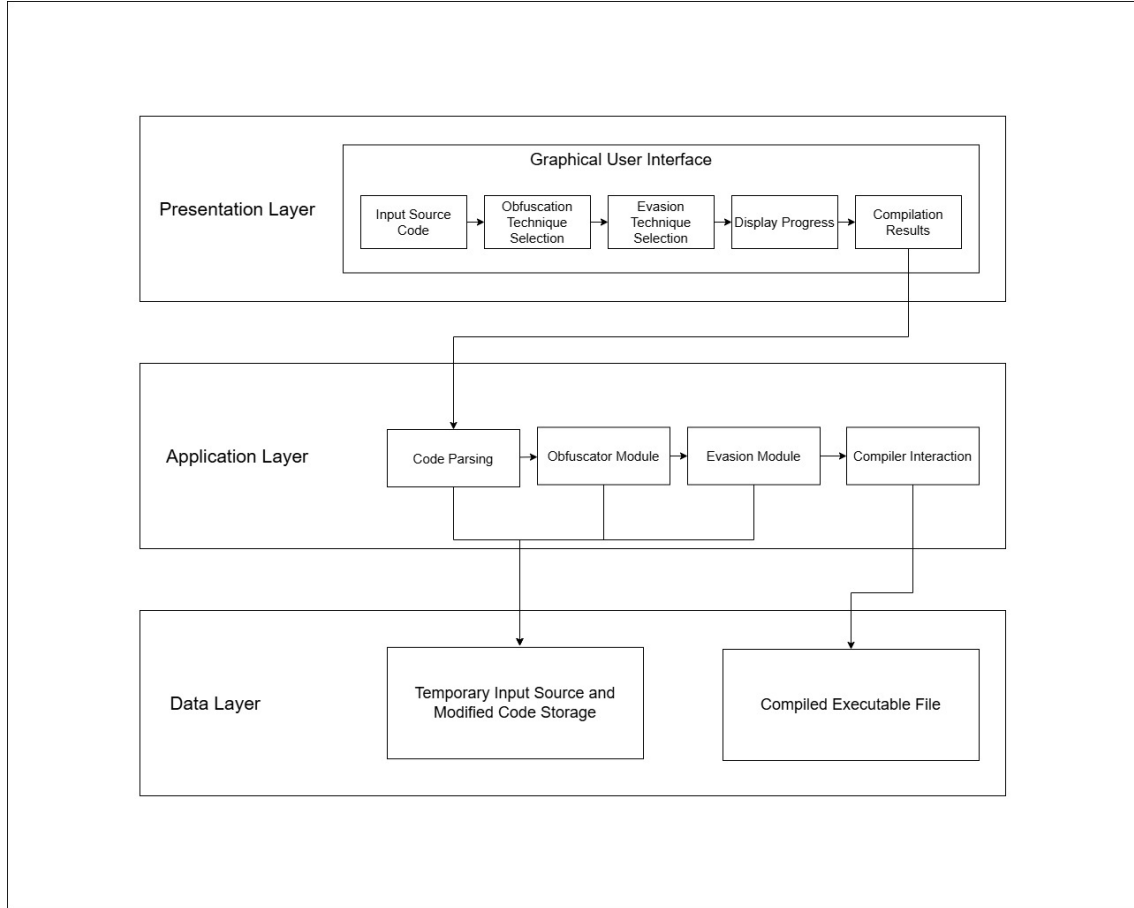


Figure 3.2: Nether's Gate Layered Architecture Diagram

A layered approach on data storage, application logic, and the user interface makes the system easy to understand, maintain and extend.

3.2 Design Models

This part will describe designs models suitable for system use through a procedural approach. They integrate various processes, how to structure them, how to manage their flow, and the way they interact with each other in an easy to grasp and visual way. Purpose and how the system operates will be briefly described for each model.

1. **Activity Diagram:** The Activity Diagram show the workflow of key works for the system. It shows you how the system moves from one activity to another. Each arrow denotes the flow between them and each activity node represents a single step in the operation of the system.

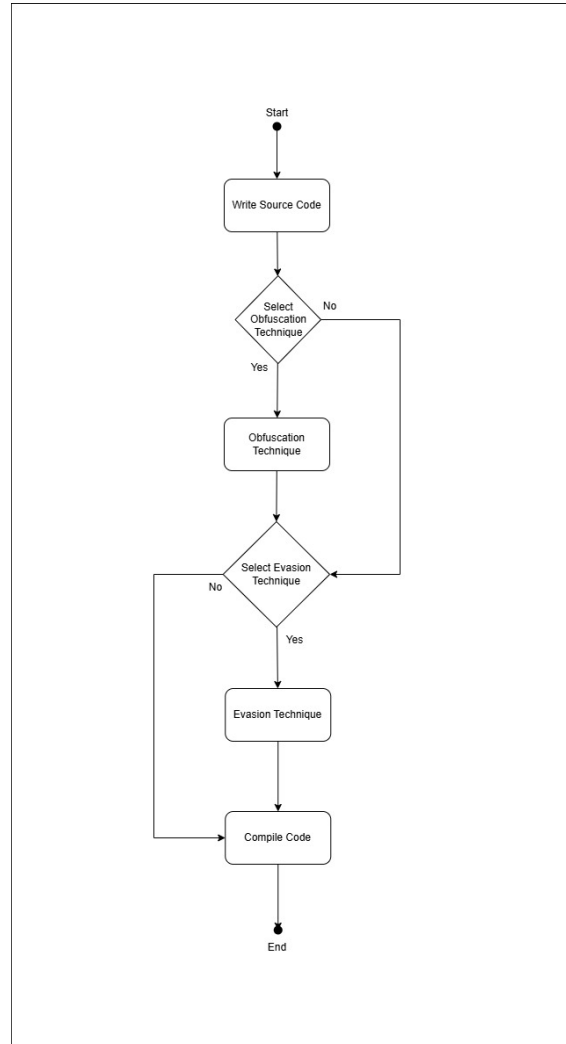


Figure 3.3: Nether's Gate Activity Diagram

2. **Data Flow Diagram:** The Data Flow Diagram (DFD) describes what data passes through the system. It involves data of information transfer between processes, data stores and external entities (sources, sinks). To get a more granular view of the system's data flows, the DFD is extended to Level 2.

Level 0 (Context Diagram): Provides an overview of the entire system, showing the interaction between the system and external entities.

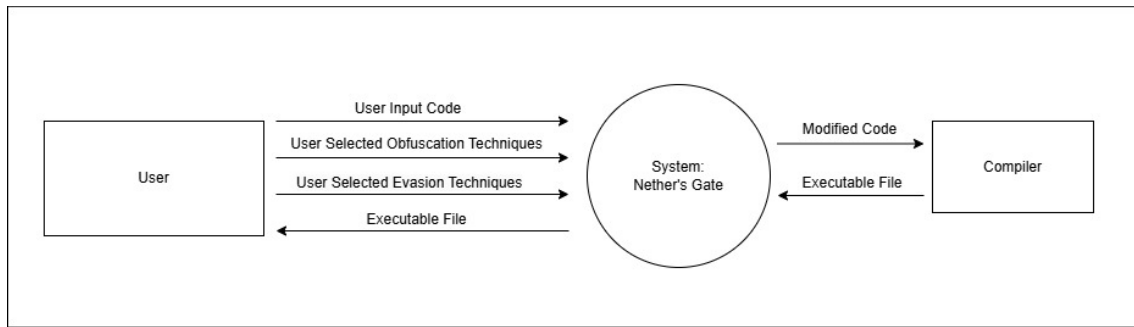


Figure 3.4: Nether's Gate DFD Level 0 Diagram

Level 1: Decomposes the major processes of the system, showing more detail about how data is processed.

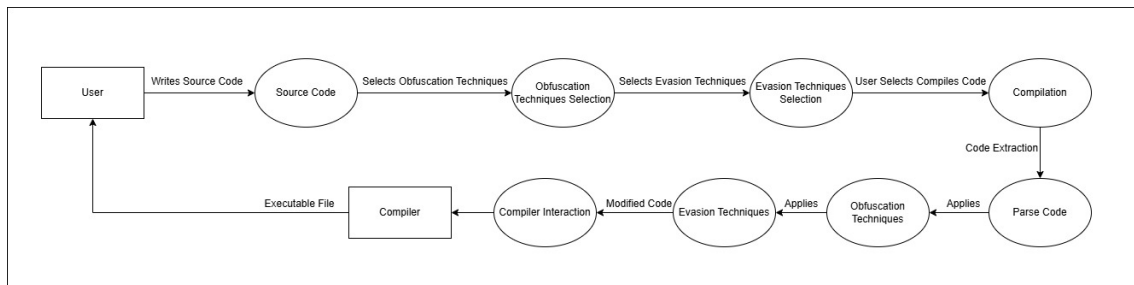


Figure 3.5: Nether's Gate DFD Level 1 Diagram

- 3. System-level Sequence Diagram:** The System-level Sequence Diagram captures the interaction between different components of the system over time. It shows how objects (or entities) within the system interact with each other through message passing, especially focusing on the order in which these interactions occur.

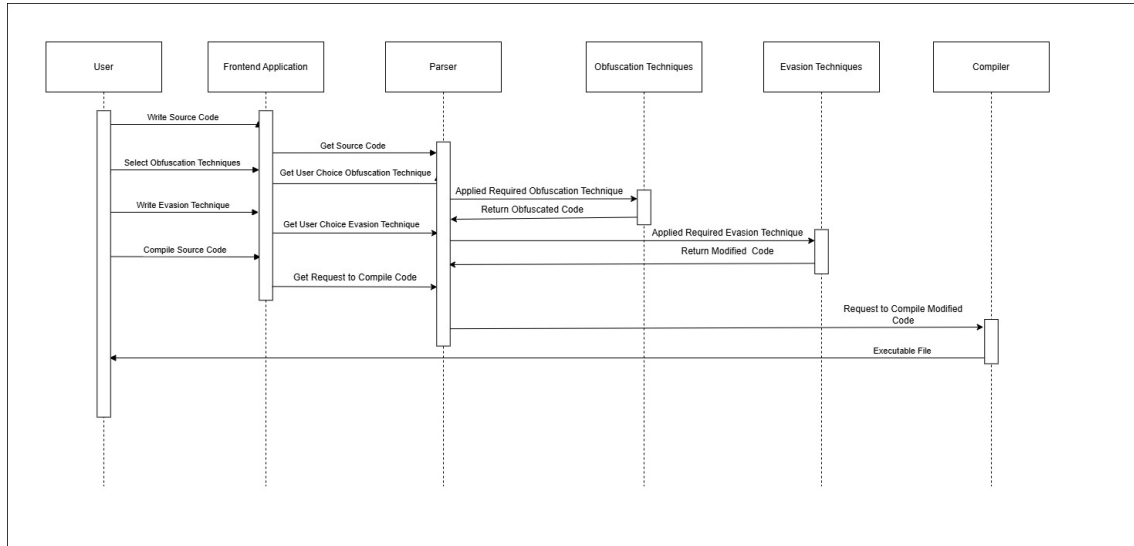


Figure 3.6: Nether's Gate System Level Sequence Diagram

Bibliography

- [1] Sebastien Andrivet and Lion Krischer. Obfuscation library based on c++11/14 and metaprogramming, 2020. Available at GitHub: <https://github.com/andrivet/ADVobfuscator>.
- [2] James Forshaw. *Windows Security Internals: A Deep Dive into Windows Authentication, Authorization, and Auditing*. Self-published, 2022.
- [3] Matt Hand. *Evading EDR: The Definitive Guide to Defeating Endpoint Detection Systems*. Self-published, 2021.
- [4] Roger S. Pressman. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, NY, 8th edition, 2014.
- [5] Ian Sommerville. *Software Engineering*. Addison-Wesley, Boston, MA, 9th edition, 2010.
- [6] Nir Yehoshua and Uriel Kosayev. *Antivirus Bypass Techniques: Learn Practical Techniques and Tactics to Combat, Bypass, and Evade Antivirus Software*. Packt Publishing Limited, Birmingham, UK, 2021.